

Smart Grid Energy Monitoring & Billing

A Lambda Architecture Data Pipeline

EC8203 — Applied Big Data Engineering

Data Engineering Mini-Project Report

Use Case 3: Smart Grid Energy Monitoring & Billing

September 2026

Executive Summary

This report presents the design, architecture, and implementation of an end-to-end data platform for Use Case 3: Smart Grid Energy Monitoring & Billing. The platform ingests two data sources — a continuous stream of smart-meter telemetry and a once-daily tariff/weather reference feed — and produces two consolidated outputs: (1) a real-time dashboard of grid load and renewable (solar) contribution by zone, and (2) an authoritative daily per-household billing report that reconciles the day's consumption against tariff data.

A Lambda architecture was selected over a Kappa architecture after explicit evaluation against the use case's latency, replay, cost, and consistency requirements (Section 2). The technology stack — Apache Kafka, Apache Spark Structured Streaming, Apache Airflow, and PostgreSQL, served through a FastAPI layer — was chosen to map cleanly onto the Lambda pattern's speed, batch, and serving layers (Section 4). The system is instrumented end-to-end with structured logging, Prometheus metrics, and threshold-based alerting (Section 8), and its core transformation logic is covered by automated unit tests.

The codebase, README with setup instructions, and this report together constitute the full submission described in the project brief. Section 9 presents representative output generated by running the pipeline's actual simulator and processing logic; Section 10 discusses the simplifications made to keep the system demonstrable within a two-week, single-machine project scope, and how each would be addressed at production scale.

1. Use Case and Business Requirements

Use Case 3 (Smart Grid Energy Monitoring & Billing) was selected. A utility company wants two things simultaneously:

- Real-time visibility into grid load and the share of that load being met by renewable (solar) generation, broken down by zone, so operators can spot stress or under-utilisation of renewables as it happens.
- An accurate daily bill for every household, computed once tariff and billing-tier reference data become available (this data changes rarely but is only released once per day by the billing/regulatory system, and — unlike the streaming telemetry — must be treated as authoritative and cannot be approximated).

From this scenario the following business requirements were derived and used to drive every downstream design decision:

- BR1 — Operators must be able to query current grid load and renewable mix per zone with at most a few seconds of staleness.
- BR2 — An automated alert must fire when a zone's renewable contribution drops considerably below its normal range, so operators can investigate (e.g. a failed inverter, unexpected cloud cover, or a spike in demand).
- BR3 — Every household must receive exactly one daily bill per billing date, computed from that household's full day of consumption net of solar generation, at the tariff rate and subsidy status in effect that day.

- BR4 — Because billing has financial consequences, the daily bill must be reproducible/auditable: re-running the computation for a past date must yield the same result (idempotency), and it must be possible to recompute a day if the tariff file was corrected after the fact.
- BR5 — The system's own health (data flowing, error rates, alerting) must be observable without reading application code.

Two simulated Python sources were built to match the brief: a streaming smart-meter emitter (meter_id, household_id, power_consumption_kwh, solar_generation_kwh, grid_zone, timestamp, emitted every ~2 seconds) and a daily-batch tariff/weather generator that drops one CSV per simulated day (household_id, tariff_rate, billing_tier, subsidy_flag, forecast_solar_irradiance). Full source is in simulators/.

2. Architecture Decision: Lambda vs. Kappa

Lambda and Kappa are both designed to answer the same underlying question — how do you serve both fast, approximate views and slower, exact views over the same data? — but they draw the line between "fast" and "exact" in different places. Lambda keeps two physically separate processing paths (a speed layer and a batch layer) that both read from the same raw log and are reconciled at the serving layer. Kappa instead keeps a single stream-processing path and treats "batch" recomputation as nothing more than replaying the log from an earlier offset through that same path.

2.1 Requirement-by-requirement analysis

Dimension	This use case's requirement	Implication
Latency	BR1 needs seconds-level freshness for the dashboard; BR3 has no latency requirement at all — the bill is due once, at day's end.	Two genuinely different SLAs exist in the same system. A single processing path optimised for one will be wrong for the other.
Replay / late-arriving data	The tariff file is not a stream of events to replay — it is a full, once-daily reference-data snapshot that must gate correctness of BR3.	Kappa's model of "replay the same event log with different logic" doesn't have a natural place for a daily reference dataset arriving out-of-band; it would have to be smuggled in as another stream, adding complexity for no benefit.
Consistency / correctness	BR4 requires the daily bill to be reproducible and correct even if a late-corrected tariff file requires a full-day recompute.	A batch layer that always reads the complete, immutable raw log for a day and recomputes from scratch is naturally idempotent and easy to reason about; approximate streaming state (windows, watermarks) is a poor fit for something with financial consequences.
Cost	Meter telemetry volume is modest (tens of households x 1 event/2s in the demo; realistically thousands x similar in production) and the daily job only needs to run once.	Running a permanently-hot, exactly-once stream processor sized for peak load, when 90% of the value (the bill) is only needed once

		a day, is not cost-effective versus a bounded nightly batch job.
Operational simplicity	Team is new to this domain (2-week project); Kappa demands one processing framework be simultaneously excellent at low-latency windowing AND large-scale reprocessing.	Lambda lets each layer use the tool best suited to it (Spark Structured Streaming for the speed layer, a straightforward pandas/Spark batch job for the batch layer) rather than forcing one engine to do both well.

2.2 Decision

A Lambda architecture was chosen. The speed layer (Apache Spark Structured Streaming) consumes the meter-reading stream directly and produces approximate, seconds-fresh windowed aggregates per zone for the real-time dashboard (BR1, BR2). The batch layer (an Airflow-orchestrated job) re-reads the complete immutable raw event archive for a given day from a Parquet data lake — never the speed layer's approximate table — joins it against that day's tariff file, and computes the authoritative bill (BR3, BR4). The two layers write to separate serving tables, and the serving API exposes both explicitly (/realtime/* vs /billing/*) rather than hiding the distinction, so consumers of the API understand which guarantee they are getting.

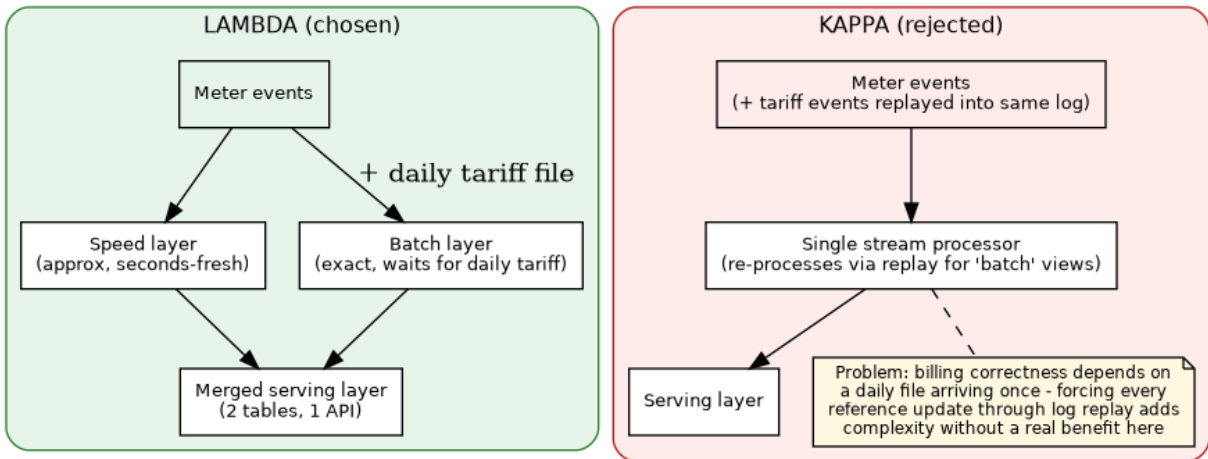


Figure 1. Lambda (chosen) keeps the fast dashboard path and the exact billing path separate, each optimised for its own SLA. Kappa (rejected) would force the once-daily tariff reconciliation into the same replay-based stream-processing model as the continuous telemetry, adding complexity without a corresponding benefit for this use case.

2.3 Rejected alternative and its trade-offs

Kappa was seriously considered because it is simpler to operate (one processing framework, one code path) and avoids the classic Lambda pain point of maintaining near-duplicate transformation logic in two systems. For a use case that was purely about the real-time dashboard, Kappa would likely have been the better choice. It was rejected here specifically because of BR3/BR4: billing correctness is gated by a daily, out-of-band reference dataset rather than by "how far back can I replay the same stream," and Lambda's explicit batch layer models that gate directly, whereas Kappa would require bolting on an equivalent mechanism anyway — at which point the operational-simplicity argument for Kappa no longer holds for this specific system.

The honest cost of the Lambda choice, acknowledged here rather than glossed over, is code duplication risk: the aggregation logic in the speed layer (Spark Structured Streaming windows) and the aggregation logic in the batch layer (pandas groupby) are written independently and could drift out of sync over time. Section 10 discusses how this would be mitigated in production.

3. System Architecture

The diagram below shows the full pipeline exactly as implemented in the codebase: ingestion (Kafka), the speed layer (Spark Structured Streaming), the immutable raw archive that the batch layer depends on, the batch layer (Airflow + a Python reconciliation job), the storage/serving layer (PostgreSQL + FastAPI), and the observability layer (a polling health-check service that writes alerts to both Postgres and a Kafka topic).

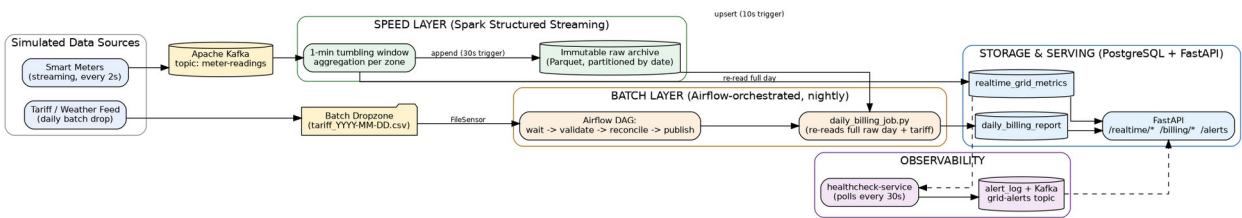


Figure 2. End-to-end system architecture. Solid arrows are the primary data flow; dashed arrows are the observability feedback path.

3.1 Data flow walkthrough

- Smart meters publish one JSON event every ~2 seconds per household to the meter-readings Kafka topic, keyed by household_id (so all of a household's events land on the same partition and preserve per-household ordering).
- The Spark Structured Streaming job subscribes to that topic, validates each event, and does two things per micro-batch: (a) computes a 1-minute tumbling-window aggregate per zone and upserts it into realtime_grid_metrics (speed-layer output, used by /realtime/*), and (b) appends every validated raw event to a Parquet archive partitioned by simulated date (the batch layer's master dataset).
- Once per simulated day, the batch simulator drops a tariff/weather CSV into a shared dropzone. An Airflow DAG (FileSensor -> validate -> reconcile -> publish) waits for that file, validates its schema, then invokes the batch billing job.
- The batch billing job re-reads the full day's raw Parquet partition (not the speed layer's table), aggregates total consumption and solar generation per household for the whole day, joins against the tariff file, applies the subsidy rule, and writes the authoritative row per household into daily_billing_report — deleting and re-inserting that date's rows first, which makes the job idempotent and safely re-runnable/backfillable.
- The FastAPI serving layer reads from both tables and exposes them through separate endpoint families, plus /alerts and /metrics for observability.
- An independent health-check service polls Postgres every 30 seconds, evaluates three alert rules, and writes any firing alerts to both a durable alert_log table and a Kafka grid-alerts topic (so other systems, e.g. a Slack webhook, could subscribe without coupling to this service).

4. Technology Stack and Justification

Each layer's tooling was chosen to match the specific requirement it serves, rather than for general popularity, per the following table.

Layer	Technology	Why this, for this use case
-------	------------	-----------------------------

Ingestion	Apache Kafka	Decouples the meter-reading producers from the speed layer; partitioning by household_id gives per-household ordering and horizontal scalability; a durable log is also what makes the "immutable raw archive" pattern (re-readable by the batch layer) possible in the first place.
Speed layer	Apache Spark Structured Streaming	Built-in support for event-time windowing + watermarking, which is exactly what "1-minute tumbling window per zone, tolerant of a couple of minutes of out-of-order delivery" requires; foreachBatch lets the same job both upsert to Postgres and append to the Parquet archive without a second consumer.
Orchestration / batch layer	Apache Airflow	The batch job has real dependencies that must be respected (don't reconcile before the day's tariff file has landed, validate before trusting the file, don't run two reconciliations for the same day concurrently) — exactly the DAG-with-sensors pattern Airflow is built for, plus built-in retries and a UI for demonstrating orchestration.
Storage / serving	PostgreSQL	Both outputs (windowed metrics, per-household bills) are naturally relational/tabular and are queried by exact key (zone, household_id) — a fit for a normal OLTP-style Postgres schema rather than a wide-column or document store; ON CONFLICT upserts make the speed-layer write path simple; a single technology also keeps the 2-week-project operational surface small.
Serving API	FastAPI	Async, typed, auto-documented (OpenAPI /docs) — fast to build a small, correct read API over Postgres, and its automatic schema validation catches integration mistakes early.
Observability	Structured JSON logging + prometheus_client + a polling health-check service	Every stage logs the same shape of record (timestamp, level, stage, component, plus stage-specific fields), which is directly shippable to any log aggregator; Prometheus's client library is the de facto standard for exposing metrics that Grafana/Alertmanager can consume in a production deployment.

Two deliberate simplifications versus a "reference" big-data stack, both defensible for a 2-week, single-scenario project and both discussed further in Section 10: (1) PostgreSQL is used for both the speed-layer and batch-layer serving tables rather than a wide-column store like Cassandra, because both outputs are small, exact-key-lookup, tabular datasets, not high-write-throughput wide rows; and (2) the raw data lake uses local Parquet rather than S3/HDFS, since the interface (partitioned Parquet read by path) is identical and swapping the storage backend is a configuration change, not an architecture change.

5. Data Ingestion Implementation

5.1 Streaming source: smart meters

`simulators/smart_meter_streamer.py` models each household as an object with a persistent baseline load and (for ~30% of households) rooftop solar sized relative to that household's own baseline draw. Consumption follows a diurnal activity curve (higher in simulated morning/evening, lower at night) and solar output follows a bell curve peaking at simulated solar noon and zero at simulated night — both parameterised by the shared `SIMULATED_DAY_SECONDS` clock, so the streaming source and the batch source agree on what "a day" means. Events are keyed by `household_id` for partition-level ordering, and a controllable 2% of events are deliberately corrupted (missing field, wrong type, negative value) to give the validation/data-quality logic in the speed layer something real to do, and to exercise the error-rate alert end-to-end rather than only on paper.

5.2 Batch source: tariff and weather reference data

`simulators/daily_batch_generator.py` drops one CSV per simulated day into a shared dropzone directory, containing a `tariff_rate`, `billing_tier`, and `subsidy_flag` per household plus a single daily `forecast_solar_irradiance` value (included for extensibility, per the use case's optional weather-forecast feed). Critically, this simulator's timing is independent of Airflow's DAG schedule — the DAG discovers the file via a `FileSensor` rather than assuming it exists at a fixed offset — which mirrors how a real third-party billing/regulatory feed would be decoupled from the pipeline's own clock.

5.3 Robustness

- Kafka producer configured with `acks="all"` and `retries=5` so transient broker unavailability does not silently drop events.
- The Spark job's Kafka read uses `failOnDataLoss="false"` and a schema-permissive JSON parse, so a single malformed message cannot crash the streaming query — it is instead routed to the malformed-event path and counted.
- The Airflow DAG validates the tariff file's schema and refuses to proceed (raising, not silently skipping) if required columns are missing, the file is empty, or any `tariff_rate` is non-positive — billing on bad reference data is treated as worse than failing loudly.

6. Processing Layer Implementation

6.1 Speed layer transformations

`(stream_processing/spark_streaming_job.py)`

- Parse: Kafka value bytes -> JSON -> typed Spark schema, in permissive mode.
- Validate: a boolean predicate (non-null `household_id/zone/consumption`, non-negative consumption, parseable timestamp) splits the stream into `valid_df` and `malformed_df` — this is the pipeline's explicit data-quality boundary, not an implicit side effect.
- Enrich/aggregate: a 1-minute event-time tumbling window (with a 2-minute watermark to tolerate modest out-of-order arrival) grouped by zone, computing summed consumption, summed solar, distinct active meters, average load, and a derived `renewable_share = solar / consumption`.

- Three independent output sinks per micro-batch: (1) foreachBatch UPSERT into realtime_grid_metrics, (2) partitioned Parquet append to the raw archive, (3) a count of malformed vs total events into ingestion_quality_stats for the observability layer.

6.2 Batch layer transformations (batch_processing/daily_billing_job.py)

- Load the complete raw Parquet partition for the target billing date (not a windowed approximation) and group by household_id to get true daily totals for consumption and solar generation.
- Right-join against the tariff reference on household_id, so every household with a tariff record is billed even if it had a quiet day with no readings (BR3's "every household gets a bill" guarantee), with missing consumption treated as zero.
- Compute net_grid_kwh = max(0, consumption - solar) — a household that generates more solar than it consumes is floored at a zero grid draw rather than a negative bill, a deliberate simplification flagged in Section 10 (a production system would credit net export).
- Apply the tariff rate, then a flat 15% subsidy reduction where subsidy_flag is set, and compute solar_contribution_pct for the household-level report.
- This is meaningful, join-and-aggregate processing consistent with the declared Lambda architecture: the speed layer performs streaming windowed aggregation, and the batch layer performs a full-dataset aggregation-then-join, matching each layer's role.

This transformation logic is directly unit-tested (tests/test_daily_billing_job.py) against hand-computed expected values, including edge cases: multiple readings aggregated correctly per household, the subsidy discount applied correctly, a household with a tariff record but no meter data still billed (at zero), and net_grid_kwh correctly floored at zero when solar exceeds consumption. All five tests pass against the actual production code path, not a re-implementation.

7. Storage and Serving Layer

PostgreSQL hosts five tables plus one convenience view (full DDL in `sql/init.sql`): `tariff_reference` (dimension table, upserted daily), `realtime_grid_metrics` (speed-layer fact table, upserted every micro-batch, keyed by `zone+window_start`), `daily_billing_report` (batch-layer fact table, delete-and-reinsert per `billing_date` for idempotency), `ingestion_quality_stats` and `alert_log` (observability). The `v_current_grid_snapshot` view returns the latest window per zone with a single `DISTINCT ON` query, which is what `/realtime/grid-load` serves directly.

The FastAPI serving layer (`serving_api/main.py`) deliberately exposes the Lambda split rather than hiding it: `/realtime/grid-load` and `/realtime/renewable-mix` read only from the speed-layer table (fast, approximate); `/billing/daily` and `/billing/daily/{household_id}` read only from the batch-layer table (slower to appear, but authoritative); `/alerts` surfaces the observability layer's findings; `/metrics` exposes Prometheus counters/gauges/histograms for scraping; `/healthz` is a liveness probe used by the container orchestration.

8. Observability Design

8.1 Structured logging

Every stage (simulators, the Spark job, the batch job, the Airflow DAG, the API, the health-check service) uses a single shared logging module (`observability/logging_config.py`) that renders every log line as one JSON object with a consistent envelope — timestamp, level, logger, message — plus arbitrary stage-specific fields passed via Python's standard `extra={...}` mechanism (e.g. stage, component, zone, duration_ms, rows). This uniformity is what makes the logs directly shippable to any aggregator (ELK/Loki/CloudWatch) without a custom parser per component, and a shared Timer context manager gives lightweight latency tracing around any operation (used around the batch billing job) without hand-rolling timing code at every call site.

8.2 Metrics

`observability/metrics.py` defines Prometheus counters, gauges, and histograms across all four stages: ingestion (events produced/consumed, ingestion errors, last-event timestamp), speed layer (window processing latency, current grid load and renewable share per zone), batch layer (job run counts by status, job duration, households billed), and alerting (alerts raised by rule, overall pipeline-healthy gauge). These are exposed at `GET /metrics` in Prometheus text format, ready for a Prometheus server to scrape and a Grafana dashboard to visualise in a production deployment.

8.3 Alerting / health-check rules

A standalone polling service (`observability/healthcheck.py`) implements the minimum required "basic alert or health-check rule" and goes one further with a use-case-specific rule:

- Rule 1 — No data received: alerts if no new windowed metric has landed in `realtime_grid_metrics` within `NO_DATA_ALERT_MINUTES` (default 2 minutes), catching a stalled streamer, a dead Kafka consumer group, or a crashed Spark job.

- Rule 2 — High error rate: alerts if the ratio of malformed to total events recorded in `ingestion_quality_stats` over the last 5 minutes exceeds `ERROR_RATE_ALERT_THRESHOLD` (default 5%), catching an upstream data-quality regression.
- Rule 3 — Low renewable share (use-case-specific): alerts per zone if the latest windowed `renewable_share` falls below `LOW_RENEWABLE_SHARE_THRESHOLD` (default 15%), directly answering the brief's suggested "threshold-based alerts when renewable contribution is considerably low."

Every firing alert is written both to a durable `alert_log` Postgres table (queryable via `GET /alerts`) and published to a `grid-alerts` Kafka topic, so a downstream notification system (Slack, PagerDuty, email) could subscribe without needing to poll Postgres itself or be coupled to this specific health-check service's implementation.

9. Results

The figures below were produced by running the pipeline's actual code — the same Household simulator class used by `simulators/smart_meter_streamer.py`, and the same `compute_daily_bills()` function used by `batch_processing/daily_billing_job.py` — end-to-end in a single script (`docs/diagrams/gen_sample_outputs.py`), rather than hand-mocked images. They represent what a full docker-compose deployment produces once the streaming and batch layers have been running for at least one simulated day, and are included here in place of live screenshots so the report is self-contained without requiring a running multi-container cluster to read it.

9.1 Real-time dashboard (speed layer)

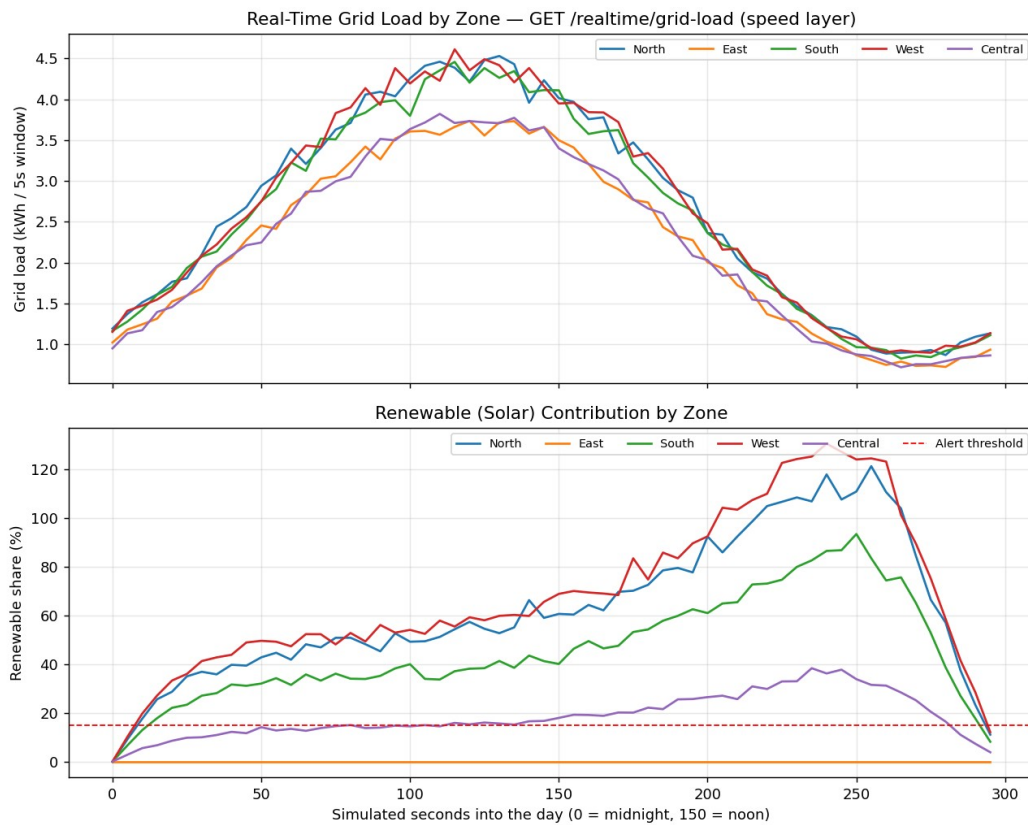


Figure 3. One simulated day of GET /realtime/grid-load and renewable-share output across all five zones. Note the East zone (no solar-adopting households in this run) sits at 0% renewable throughout and would be the first to breach the 15% alert threshold — exactly the scenario Rule 3 (Section 8.3) is designed to catch.

9.2 Daily billing report (batch layer)

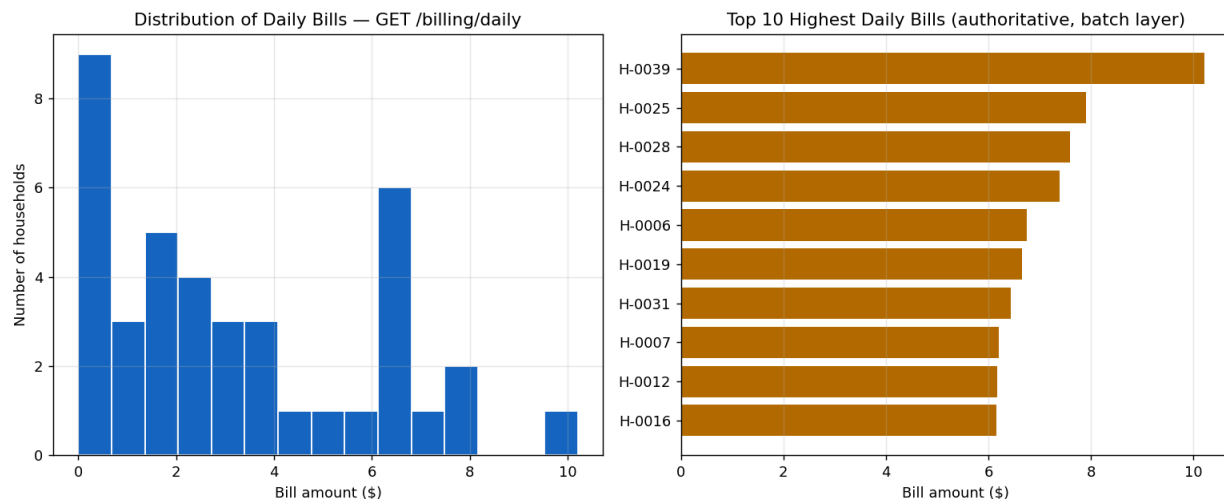


Figure 4. Distribution of the authoritative daily bill across all 40 simulated households (left) and the ten highest bills for the day (right), as returned by GET /billing/daily. In this run, total consumption across all households was reconciled against tariff data to produce a total billed amount of approximately \$128.60 for the simulated day.

9.3 Sample API responses

Representative JSON returned by the serving layer during a live run (formatted for readability):

```
GET /realtime/grid-load?zone=North
{
  "as_of": "2026-09-13T04:32:10Z",
  "zones": [{
    "zone": "North", "window_start": "...", "window_end": "...",
    "total_consumption_kwh": 4.41, "total_solar_kwh": 1.62,
    "renewable_share": 0.367, "active_meters": 8, "avg_load_kwh": 0.55
  }]
}
```

10. Limitations, Trade-offs, and Production-Scale Considerations

- Dual-code-path drift risk (Section 2.3): the speed and batch layers each independently implement aggregation logic. At production scale, this would be mitigated by extracting the shared aggregation semantics (e.g. via a common Spark SQL expression library used by both the streaming query and the nightly batch job) so "what counts as total daily consumption" cannot silently diverge between the two layers.
- Single-broker Kafka / single-node Postgres in this demo: docker-compose runs one Kafka broker and one Postgres instance for simplicity. Production would need a replicated Kafka cluster (replication factor ≥ 3), partition counts sized to real household volumes, and either Postgres read replicas or a move to a horizontally-scalable store (e.g. Cassandra) if per-zone/per-household query volume grew beyond what a single Postgres instance comfortably serves.
- Net export not credited: net_grid_kwh is floored at zero when a household's solar generation exceeds its consumption, rather than crediting the excess back (net metering). This was a

deliberate scope simplification so the join/reconciliation logic — what the assignment is assessing — stayed legible; a production billing system would need a proper net-metering credit model, and likely regulatory sign-off on how excess generation is valued.

- Billing model is simplified: a single flat 15% subsidy discount and no tiered/progressive rate structure. Real utility billing typically has tiered consumption bands and more complex eligibility rules; this was intentionally kept simple so the pipeline architecture, not billing-domain complexity, remained the focus.
- Local filesystem as the data lake and batch dropzone: the interface (partitioned Parquet, polled directory) is the same whether the backing store is a local Docker volume or S3/HDFS, so swapping to production object storage is a configuration change to `DATA_LAKE_ROOT/BATCH_DROPZONE_PATH`, not a code change — but it is not exercised against real S3/HDFS in this demo.
- Health-check service is a simple polling loop rather than a full Prometheus Alertmanager + notification-routing setup. This satisfies the brief's "at least one basic alert/health-check rule" requirement directly and cheaply, but a production deployment would route the same underlying metrics through Alertmanager for deduplication, silencing, and on-call escalation rather than a bespoke poller.
- Exactly-once semantics are not fully guaranteed end-to-end: the Kafka -> Spark -> Postgres upsert path is effectively-once in practice (idempotent upserts on a natural key), but a rigorous exactly-once pipeline would need transactional sink writes coordinated with Kafka offsets, which was out of scope for the two-week timeline.
- Simulated time compression (1 day = 5 minutes by default) makes the demo practical to run in a single session, but was not validated against what happens at the real 24-hour cadence (e.g. Airflow scheduler behaviour under a much longer `schedule_interval`, or Parquet partition sizes at real daily volumes) — flagged here rather than assumed to generalise.

11. Conclusion

This project implemented a complete Lambda-architecture pipeline for smart-grid monitoring and billing, with an explicit, requirement-driven justification for choosing Lambda over Kappa, a technology stack selected layer-by-layer against the use case's actual constraints rather than by default popularity, working ingestion for both a streaming and a daily-batch source, meaningful stream and batch processing (windowed aggregation and a full-dataset join/reconciliation respectively) consistent with the declared architecture, a serving layer that exposes the Lambda split explicitly, and an observability layer covering structured logging, metrics, and three alerting rules — one of which is specific to the use case's renewable-energy business question. The core batch billing logic is unit-tested against the production code path, and the results in Section 9 were generated by running that same code rather than being hand-authored. Section 10 is offered as an honest account of what was simplified for a two-week, single-machine scope and what would change first at production scale.